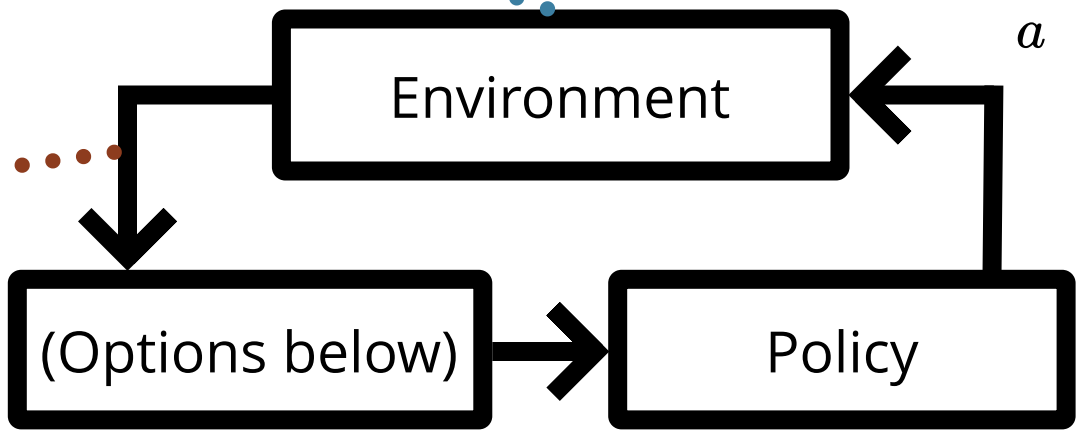
Particle Filters

POMDP Sense-Plan-Act Loop

True State

$s = TL$



Observation

$o = TL$

Option 1: History

h

History: $h_t = (b_0, a_0, o_1, a_1, \ldots a_{t-1}, o_t)$

b

Option 2: Belief Updater

Belief: $b_t = P(s_t \mid h_t)$

Review: Bayesian Filter

```
function update(b::Vector{Float64}, 𝒫, a, o)
    𝒮, T, O = 𝒫.𝒮, 𝒫.T, 𝒫.O
    b′ = similar(b)
    for (i′, s′) in enumerate(𝒮)
        po = O(a, s′, o)
        b′[i′] = po * sum(T(s, a, s′) * b[i] for (i, s) in enumerate(𝒮))
    end
    if sum(b′) ≈ 0.0
        fill!(b′, 1)
    end
    return normalize!(b′, 1)
end
```

Review: Bayesian Filter

$$b_t(s) = P(s_t = s \mid h_t)$$

```
function update(b::Vector{Float64}, 𝒫, a, o)
    𝒮, T, O = 𝒫.𝒮, 𝒫.T, 𝒫.O
    b′ = similar(b)
    for (i′, s′) in enumerate(𝒮)
        po = O(a, s′, o)
        b′[i′] = po * sum(T(s, a, s′) * b[i] for (i, s) in enumerate(𝒮))
    end
    if sum(b′) ≈ 0.0
        fill!(b′, 1)
    end
    return normalize!(b′, 1)
end
```

Review: Bayesian Filter

$$b_t(s) = P(s_t = s \mid h_t)$$

$$b' = \tau(b, a, o)$$

```
function update(b::Vector{Float64}, 𝒫, a, o)
    𝒮, T, O = 𝒫.𝒮, 𝒫.T, 𝒫.O
    b′ = similar(b)
    for (i′, s′) in enumerate(𝒮)
        po = O(a, s′, o)
        b′[i′] = po * sum(T(s, a, s′) * b[i] for (i, s) in enumerate(𝒮))
    end
    if sum(b′) ≈ 0.0
        fill!(b′, 1)
    end
    return normalize!(b′, 1)
end
```

Review: Bayesian Filter

$$b_t(s) = P(s_t = s \mid h_t)$$

$$b' = \tau(b, a, o)$$

$$b'(s') \propto Z(o \mid a, s') \sum_s T(s' \mid s, a)\, b(s)$$

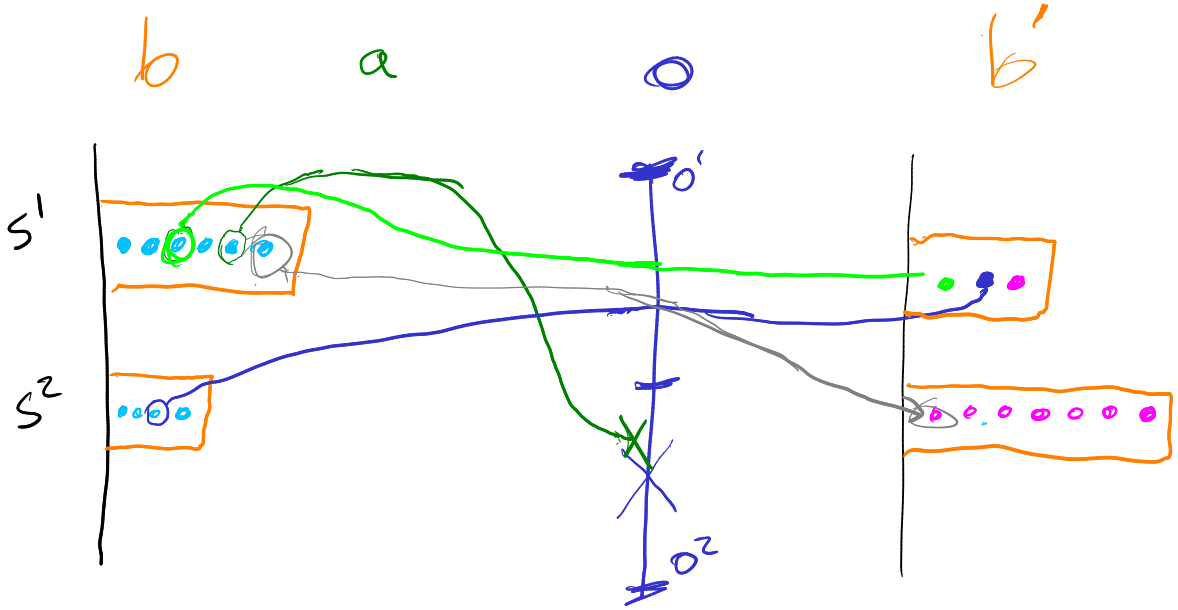
```
function update(b::Vector{Float64}, 𝒫, a, o)
    𝒮, T, O = 𝒫.𝒮, 𝒫.T, 𝒫.O
    b′ = similar(b)
    for (i′, s′) in enumerate(𝒮)
        po = O(a, s′, o)
        b′[i′] = po * sum(T(s, a, s′) * b[i] for (i, s) in enumerate(𝒮))
    end
    if sum(b′) ≈ 0.0
        fill!(b′, 1)
    end
    return normalize!(b′, 1)
end
```

$S = \{s^1, s^2\}$

Rejection Particle Filter

state

$b(s)$ · proportional to number of particles that match s



Iter 1 $G(s^1, a) \rightarrow s^2, o^2$

Iter 2 $G(s^1, a) \rightarrow s^1, o^1$

Iter 9 $G(s^1, a) \rightarrow s^2 o^1$

Algorithm 6.2 Particle filter with rejection

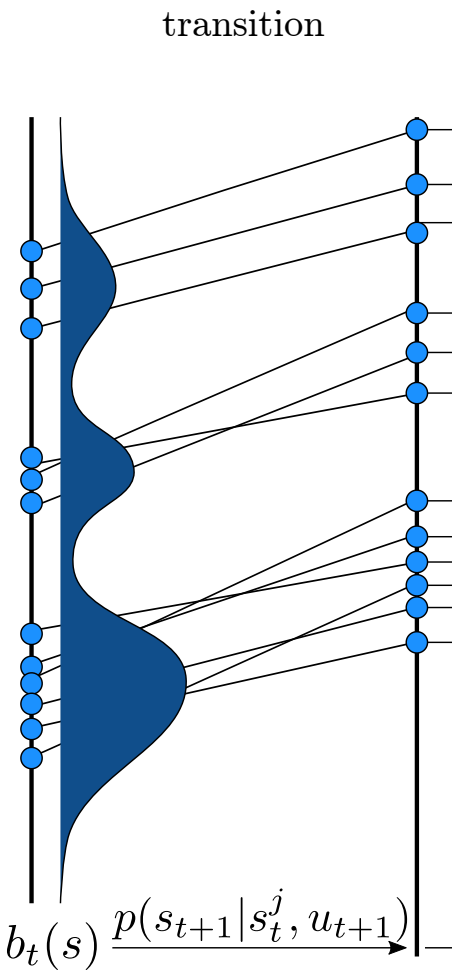
```
function UpdateBelief(b, a, o)
    b' ← ∅
    for i ← 1 to |b|
        repeat
            s ← random state in b
            (s', o') ~ G(s, a)
        until o' = o
        Add s' to b'
    return b'
```

o'

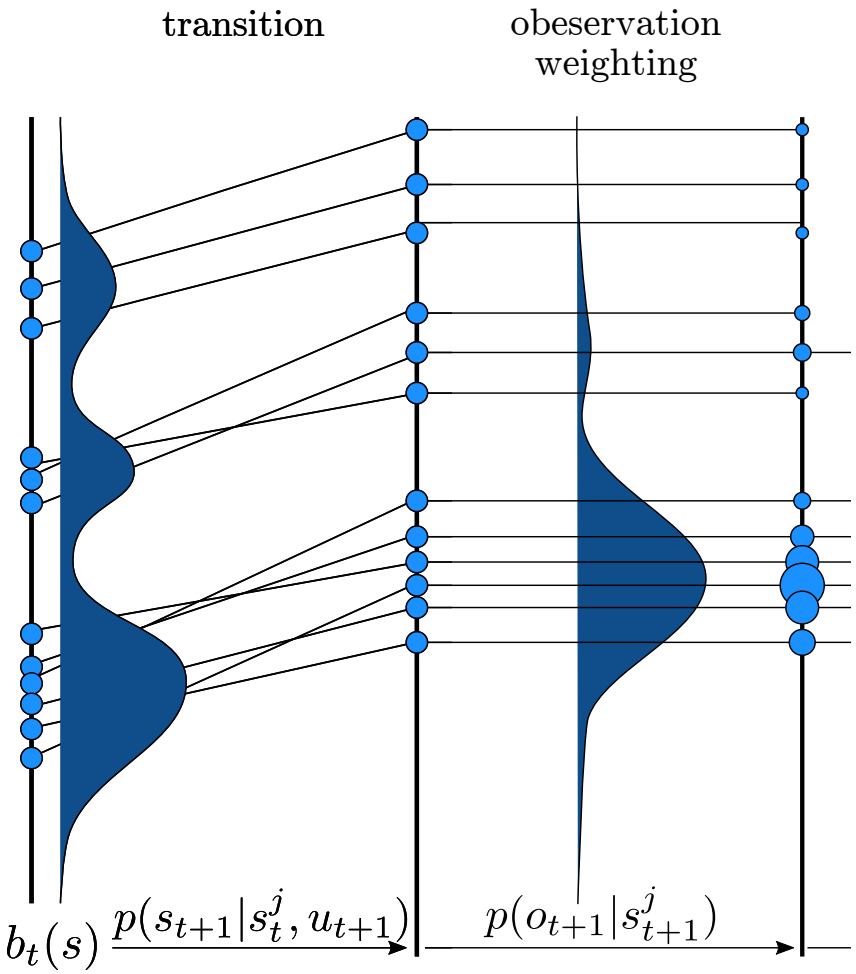
Weighted Particle Filtering

Weighted Particle Filtering

$S = \mathbb{R}$

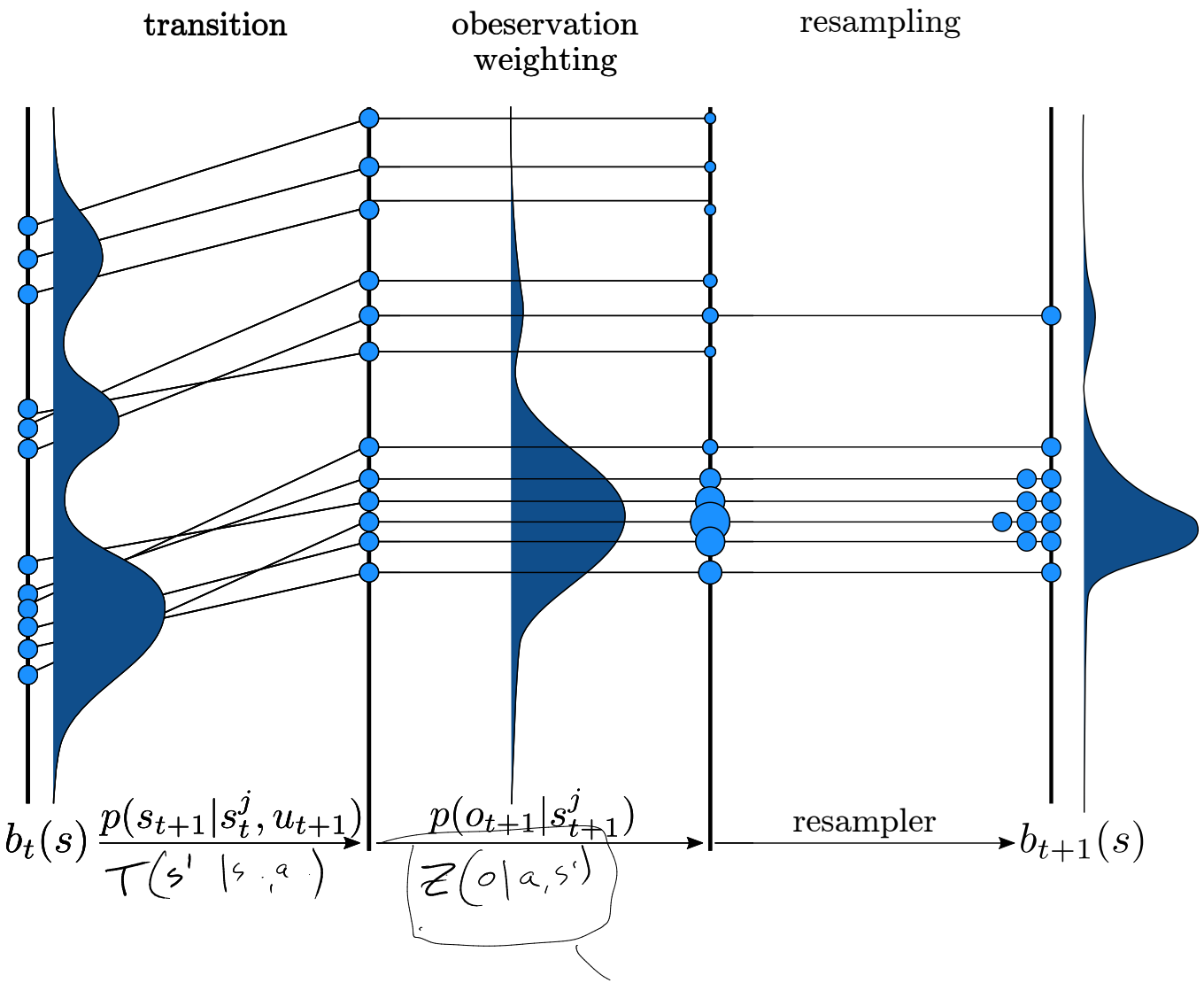


Weighted Particle Filtering

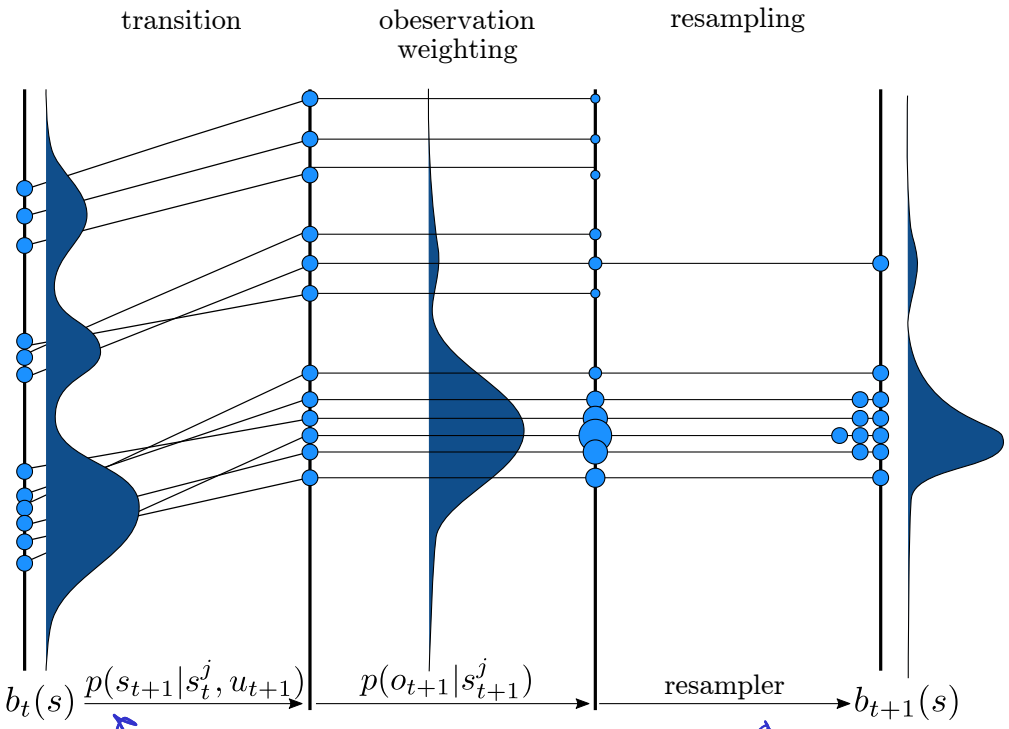


Weighted Particle Filtering

$$b(s) = \sum_{i=1}^{N} \delta(s_i = s)\, w_i$$



Weighted Particle Filtering

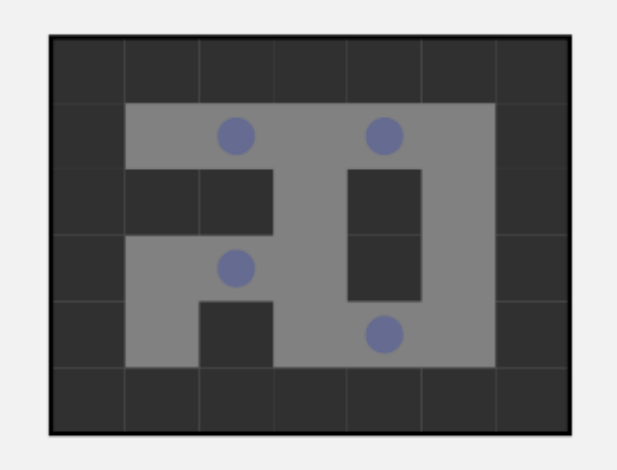


Algorithm 6.3 Particle filter without rejection

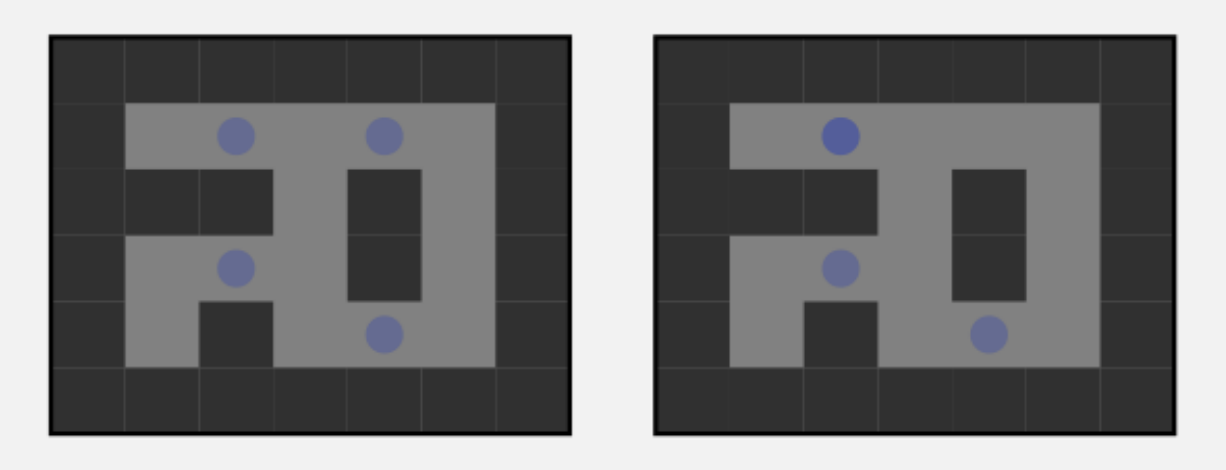
```
function UpdateBelief(b, a, o)
    b' ← ∅
    for i ← 1 to |b|
        s_i ← random state in b
        s'_i ~ G(s_i, a)
        w_i ← O(o | s'_i, a)
    for i ← 1 to |b|
        Randomly select k with probability proportional to w_k
        Add s'_k to b'
    return b'
```

Particle Depletion

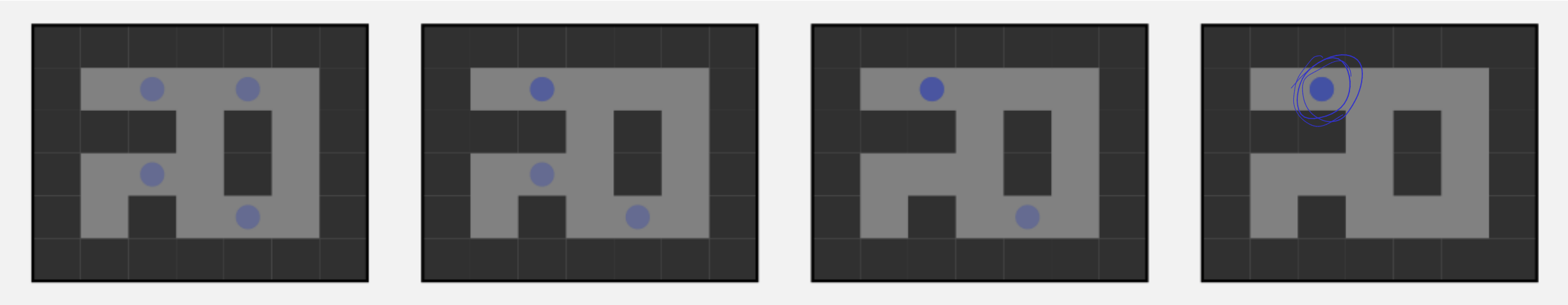
Particle Depletion



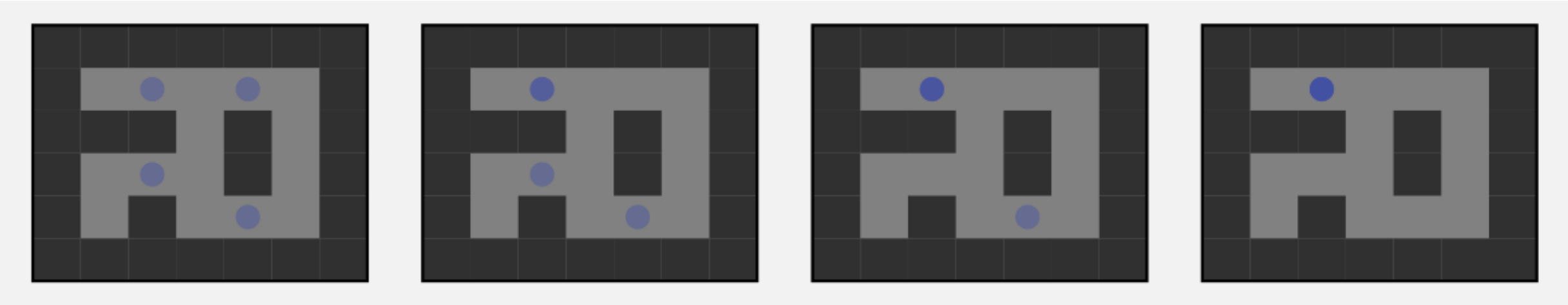
Particle Depletion



Particle Depletion

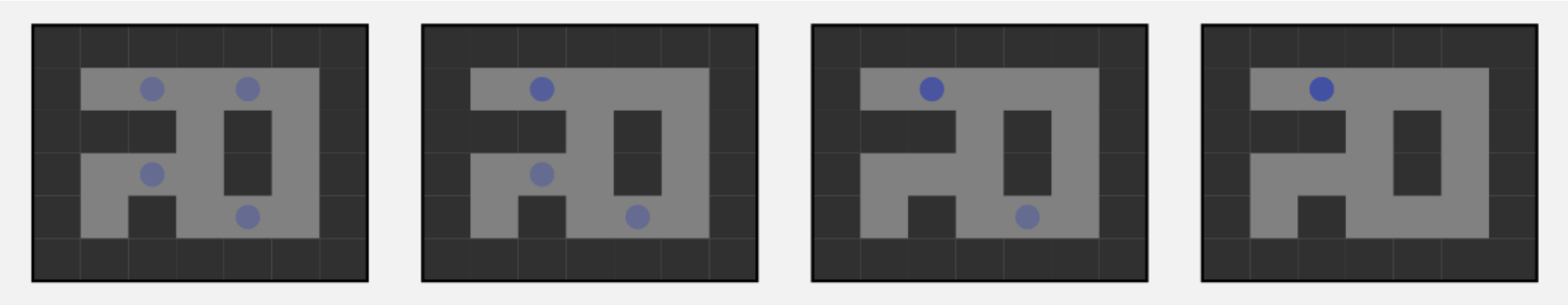


Particle Depletion



Solution: Domain specific particle injection based on:

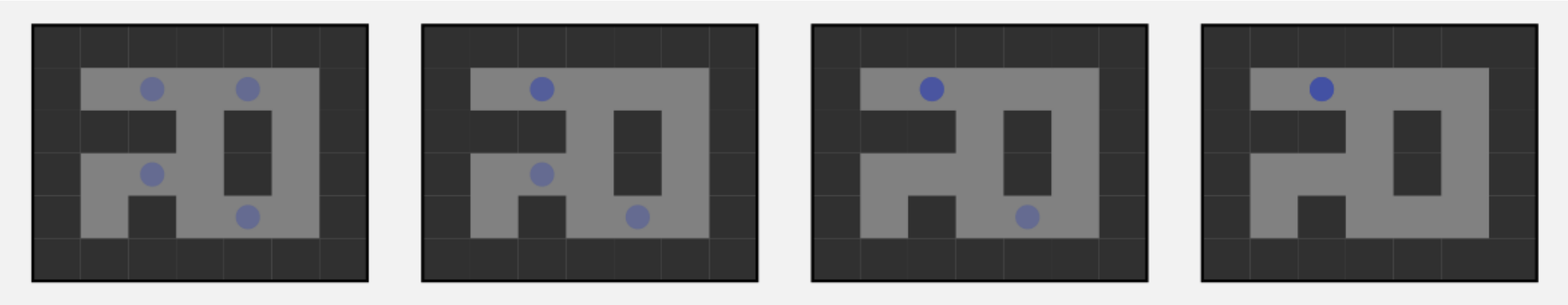
Particle Depletion



Solution: Domain specific particle injection based on:

- Weights

Particle Depletion

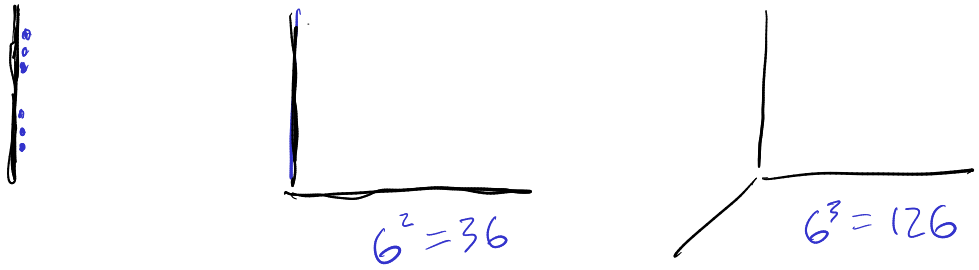


Solution: Domain specific particle injection based on:

- Weights
- Particle Diversity

Important Particle Filter Properties

- When only estimating the reward, the number of particles does **NOT** need to scale exponentially with the dimension (i.e. $n \neq k^d$)
- Implementation should have $O(n)$ complexity.



$E[R(s,a)]$

$Q_N = \frac{1}{N} \sum_i R(s_i, a)$

$\text{std}(Q_N) \propto \frac{1}{\sqrt{N}}$

Easy to have $O(n)$

Naive resampling

```
bp=[]
for i in 1:n
    push!(bp, rand(b))
end
```

$O(n^2)$

$O(n)$

Low variance resample: p. 110 of *Probabilistic Robotics* by Thrun, Burgard, and Fox